

On the influence of Test Smells on Test Coverage

Tássio Virginio

tassio.virginio@ufba.br

Federal University of Bahia (UFBA)
Salvador, BA, Brazil

Railana Santana

railana.santana@ufba.br

Federal University of Bahia (UFBA)
Salvador, BA, Brazil

Luana Almeida Martins

luana.martins1@estudante.ufla.br

Federal University of Lavras (UFLA)
Lavras, MG, Brazil

Larissa Rocha Soares

larissars@dcc.ufba.br

Federal University of Bahia (UFBA)
Salvador, BA, Brazil

Heitor Costa

heitor@ufla.br

Federal University of Lavras (UFLA)
Lavras, MG, Brazil

Ivan Machado

ivan.machado@ufba.br

Federal University of Bahia (UFBA)
Salvador, BA, Brazil

ABSTRACT

Software testing is a key practice in the software quality assurance process. Usually, the quality of a test is not analyzed before its execution, i.e., there are no tests to check the tests. When the quality of tests is not guaranteed, it may impair the quality of the software. Test Smells are an alternative to indicate problems in the test code that can affect test maintainability, more specifically readability and comprehension. This study investigates correlations between test coverage and test smells types. We also introduce the JNose Test, a tool to automate test smells detection. We analyzed 11 open source projects and detected 21 types of smells and 10 different test coverage metrics to each test class. We identified 63 out of 210 calculated correlations. Our results show that there is a relationship between test smells and test coverage, in which test smells may influence code coverage. Our findings might support software testers and help them understand the behavior and consequences of poorly written and designed tests.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; *Empirical software validation*.

ACM Reference Format:

Tássio Virginio, Railana Santana, Luana Almeida Martins, Larissa Rocha Soares, Heitor Costa, and Ivan Machado. 2019. On the influence of Test Smells on Test Coverage. In *XXXIII Brazilian Symposium on Software Engineering (SBES 2019), September 23–27, 2019, Salvador, Brazil*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3350768.3350775>

1 INTRODUCTION

Software companies face the challenge of developing complex, high-quality systems without increasing project development time and budget [14]. Usually, quality assurance is a concern throughout the software development process. However, when discussed and

specified at early stages of software development, it may avoid wasting time and resources at later stages.

Software testing plays a fundamental role in the quality assurance process [11, 17]. There are several software testing techniques, either manual or automated, among which the later has been predominantly used in software industry [20]. Although automated tests identify errors efficiently from repeated executions and with lower cost, when compared to manual test execution, they may be prone to errors [11, 17, 20]. An alternative is to plan and implement test code using good design practices [14].

The quality of automated test suites could be evaluated through the analysis of code coverage metrics, which measure the quality based on how much they cover structural elements, such as functions, instructions, branches, and conditions of a system [8]. Even with a high test code coverage, test implementation may contain poor design choices, the so-called *test smells*. The presence of test smells may negatively affect the quality of the test code [5]. Rompaey et al. [15] describe test smells as violations of the four-phase test pattern (configuration, execution, verification, and disassembly), which result in less satisfaction of the test quality criteria.

In this paper, we investigate the correlation between test smells and code coverage. From 11 software projects, we collected 21 test smells and 10 code coverage metrics. We also introduce the JNose Test tool, developed to automate the detection of test smells, and collect code coverage metrics. We aimed to analyze the correlation between each type of test smell and each type of code coverage metric. Our analysis enable to identify the existing influence of test smells on code coverage, such as the existence of strong correlations between them.

2 TEST SMELLS AND METRICS

Test code is a way to ensure the quality of the developed system [11, 17]. Tests can be either *manual*, in which the system is executed manually by a professional or *automated*, where code scripts are developed and executed when needed. Usually, automated tests present more efficient results than manual tests due to their repeatability capability, and the lack of human interference, gaining in reliability when compared to most manually developed tests [6, 7]. Thus, automated tests are predominantly used in the software industry as a safety way to modify a system in order to either improve, correct or adapt them to changes [3]. Tools for automated tests are used in order to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES 2019, September 23–27, 2019, Salvador, Brazil

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7651-8/19/09...\$15.00

<https://doi.org/10.1145/3350768.3350775>

facilitate the development of test scripts and the interpretation of their results. However, the use of those tools may not ensure the quality of the system once they might have been poorly planned [4].

Test smells are defined as bad design choices, which can negatively affect the understanding and maintenance of the test code [19]. For this study, we selected 21 types of test smells [5, 10, 13]. They are described next:

- **Assertion Roulette (AR)**: Occurs when a test method contains multiple non-documented assertions. Thus, if an assertion fails, it can be difficult to identify which one failed;
- **Conditional Test Logic (CTL)**: Occurs when a test method contains conditional expressions or loop structures. The test fails when a test statement is not executed;
- **Constructor Initialization (CI)**: Occurs when a test suit method contains a constructor;
- **Default Test (DT)**: Occurs when a test suit contains test classes created by default;
- **Dependent Test (DepT)**: Occurs when the test being executed depends on other tests' success. As a result, dependent tests can fail for the wrong reason;
- **Duplicate Assert (DA)**: Occurs when a test method tests the same condition multiple times;
- **Eager Test (ET)**: Occurs when a test method checks more than one method of the test class;
- **Empty Test (EpT)**: Occurs when a test method does not contain executable statements;
- **Exception Catching Throwing (ECT)**: Occurs when a test method is explicitly dependent on the production method throwing an exception;
- **General Fixture (GF)**: Occurs when a test case fixture is general and the test method only access part of it;
- **Ignored Test (IgT)**: Occurs when a test method is suppressed from running;
- **Lazy Test (LT)**: Occurs when several test methods check the same method;
- **Magic Number Test (MNT)**: Occurs when assert statements contain numeric literals;
- **Mystery Guest (MG)**: Occurs when a test method uses external resources (e.g., a file containing test data), and, thus, it is not self-contained. Hence, if this resource is changed, the tests can fail;
- **Print Statement (PS)**: Occurs when unit tests contain print statements;
- **Redundant Assertion (RA)**: Occurs when test methods contain assertion statements that are either always true or always false.
- **Resource Optimism (RO)**: This smell refers to a test method that makes optimistic assumptions about the existence and the state of external resources can cause non-deterministic behavior in a test result;
- **Sensitive Equality (SE)**: Occurs in test methods that contain an equality check using the method `toString`, which depends on details, such as commas and spaces. Thus, when the method for an object is changed, the test can fail;
- **Sleepy Test (ST)**: Occurs when the execution of statements in a test method needs to be paused for a certain amount

of time (e.g., simulating an external event) and, after that, it continues with the execution;

- **Unknown Test (UT)**: Occurs when a test method is written without an assertion statement, which makes it difficult to understand the purpose of the test method.
- **Verbose Test (VT)**: Occurs when the tests use a lot of code to do what they need to do. In other words, the test code is not clean and simple.

Meanwhile, software metrics are used to monitor and improve the quality of the development process and software product [18]. Coverage metrics determine how much a test suit scans a given program [9]. There are several strategies for measuring coverage, such as line coverage, branch coverage, path coverage, function/method coverage, and class coverage. Each of these strategies can be applied at any stage of testing, e.g., unit testing, integration testing, or system testing.

For this work, we select five code coverage metrics, analyzed in terms of covered and missed. **Branch** metric is related to the number of branches in a method and determines the number of executed and missed branches. Branch coverage is always available, even in the absence of debugging information in class files. **Cyclomatic complexity** is the minimum number of paths that can generate all possible paths through a method. Metrics complexity coverage and missed count the number of paths completely covered and the number of uncovered paths. **Instruction** metric provides information about the amount of code which has been either executed or lost. The amount of code is calculated independently on source formatting. **Line** coverage is calculated for the class files compiled with debug information. These metrics calculate the number of lines executed or not. Finally, **Method** metric considers the method which is executed when at least one statement was covered. Each non-abstract method contains at least one statement.

3 CORRELATION OF TESTS AND COVERAGE

This work investigates the correlation between test smells and code coverage. Thus, we define the following research question (RQ): *Is there a correlation between test smells and code coverage metrics?*

To answer the RQ, we execute the proposed JNose Test tool to analyze software projects and automatic collect test smells and code coverage metrics.

3.1 JNose Test

JNose Test is a tool developed to automatically detect test smells in test code, and to collect coverage metrics. JNose test is an extension of the Test Smell Detector [1]. Besides presenting the number of test smells detected by class, our tool shows the collection of code metrics and test coverage using the JaCoCo library¹; a unified result for all projects under analysis; and a graphical interface. In addition, the project uses the Apache Maven² to manage all library dependencies and support the compilation and execution of the JNose Test tool.

¹<https://www.eclemma.org/jacoco/>

²<https://maven.apache.org/>

Table 1: Summary of analyzed open source projects

Project	Version	LOC	JUnit LOC
<i>Joda Time</i>	2.9.7	29,272	55,554
<i>Apache Commons Text</i>	1.6	9,526	13,467
<i>Apache Commons Lang</i>	3.9	28,006	47,833
<i>Apache Commons Email</i>	1.5.1	2,947	3,989
<i>Apache Commons CSV</i>	1.6	1,787	4,783
<i>Apache Commons Colletion</i>	4.3	29,309	34,200
<i>Apache Commons Codec</i>	1.11	8,392	12,040
<i>Apache Commons DBCP</i>	2.6.0	14,458	16,846
<i>Apache Commons Math</i>	3.4.1	86,481	89,016
<i>Apache Commons Pool</i>	2.6.2	5,532	8,813
<i>Apache Commons DbUtils</i>	1.7	3,101	3,789

After executing the tool, based on the yielded results, we could investigate the correlation between the types of test smells and types of code coverage metrics values.

3.2 Data Collection and Analysis

Data collection was performed with an analysis of 21 types of smells and 10 coverage metrics. Table 1 lists the 11 projects used in data collection. They were selected using the following criteria: (i) project developed in Java language; (ii) use of Maven; (iii) implement unit tests with JUnit; (iv) available under an open source license. Projects vary in the number of lines of code (LOC). For example, the *Apache Commons CSV* project was the smallest one, with 871 LOC; and the *Apache Commons Math* was the largest one, with 30,162 LOC.

In order to describe and characterize the data sample, we used measures of central tendency and measures of dispersion. Also, we identified whether the sample data follow the normal distribution using the Kolmogorov-Smirnov (K-S) test. Since the data sample did not follow a normal distribution, we chose the Spearman correlation test to evaluate the following hypotheses:

H_0 : There is no correlation between the test coverage metrics and test smells.

H_1 : There is correlation between the test coverage metrics and test smells.

In general, our hypotheses aim to understand the relation between each metric versus each test smell collected with our tool.

4 RESULTS

Table 2 and Table 3 present the descriptive statistics for test smells and test coverage metrics, respectively. For example, regarding the LT test smell, the number of LT ranged from 0 (minimum) to 1268 (maximum). The 1st quartile represents $\frac{1}{4}$ of the data set, and it is composed only by classes without LT (zero). The 3rd quartile represents $\frac{3}{4}$ of the data set, and it is composed by classes that contain between 45 and 1268 LT. The classes analyzed presented 6 LT on average with a mean value of 50.8, and standard deviation of 120.5, indicating high dispersion of values, as Table 2 shows.

Our results shows that LT has the highest mean and the highest standard deviation. LT was followed by ET with a mean value of 8.7, and AR with 7.0. LT occurs when several test methods verify the same method. A high number of LT indicates that a method of the production class performs several functions, i.e., a method

Table 2: Descriptive Statistics for Test Smells (top 5 by Max)

Test Smells	Min	1st Qu.	Median	Mean	3st Qu.	Max.	SD
LT	0.0	0.0	6.0	50.8	45.0	1,268.0	120.5
AR	0.0	0.0	2.0	7.2	7.0	219.0	14.6
GF	0.0	0.0	0.0	2.0	0.0	196.0	12.2
ET	0.0	0.0	2.0	8.7	9.0	195.0	18.2
ECT	0.0	0.0	1.0	5.0	4.0	195.0	13.7

Table 3: Descriptive Statistics for Test Covers (top 5 by Max)

Coverage	Min	1st Qu.	Median	Mean	3st Qu.	Max	SD
InstructionC	0.0	68.0	167.0	445.3	461.0	18,146.0	1,010.9
InstructionM	0.0	0.0	5.0	34.7	25.0	2,046.0	115.9
LineC	0.0	16.0	35.0	80.0	87.0	2,010.0	142.4
BranchC	0.0	3.0	12.0	36.8	37.0	1,495.0	93.5
ComplexityC	0.0	7.0	14.0	34.1	37.0	997.0	64.5

has many responsibilities. Then, it is necessary to develop several methods to test it.

Regarding the test coverage metrics presented in Table 3, InstructionC and LineC presented the highest average values, assuming the values of 445.3 and 80.0, respectively. Also, InstructionC presented the highest dispersion, 1010.9, followed by LineC, with 142.4. These results indicate that Line and Instruction coverages tend to occur more in test smells than Branch and Complexity coverages.

Since the data follow a non-normal distribution, we applied the Spearman correlation test. The correlation was calculated between the set of test smells and the test coverage metrics analyzed, which yielded a set of 210 correlations, as Table 4 shows. The results were interpreted according to Salkind and Rainwater [16], as follows:

- 0.8 to 1.0, or -0.8 to -1.0: very strong relationship;
- 0.6 to 0.8, or -0.6 to -0.8: strong relationship;
- 0.4 to 0.6, or -0.4 to -0.6: moderate relationship;
- 0.2 to 0.4, or -0.2 to -0.4: weak relationship;
- 0.0 to 0.2, or 0.0 to -0.2: weak or no relationship.

The Correlations for the EpT, IgT, MG, PS, RO, ST, and VT, were removed from Table 4 by non-correlation. In addition, the tests of DepT and DT did not manifest themselves in the analyzed projects, so the correlation could not be calculated for these two test odors. However, the rest of the test smells have weak to strong correlation related to test coverage. In general, we identified that there is correlation between the test coverage and the test smells.

5 IMPLICATIONS FOR RESEARCH AND PRACTICE

From our correlation analysis, we highlight two main findings: (i) AR, ET, LT, and DA smells presented moderate to strong correlation with 5 test coverages, i.e., Branch, Complexity, Instruction, Method and Lines; (ii) the missing Code Coverage Metrics do not show a significant correlation with the number of smells identified in the test classes.

AR test smell is related to non-documentation of assertions, making it difficult to identify which assertion has passed or failed. We identified that there is a higher correlation between AR and method coverage (0.59) than with instruction coverage (0.41). Since the methods have more than one assertion, some of them may have comments while others may not be, leading to accounting for the method as a whole for metric coverage. Conversely, at the instruction level, only those that were not actually documented

Table 4: Correlation between Test Smells and Coverage Metrics

Smell	BranchC	BranchM	ComplexityC	ComplexityM	InstructionC	InstructionM	LineC	LineM	MethodC	MethodM
AR	0.42★	0.29❖	0.55★	0.25❖	0.41★	0.18	0.46★	0.18	0.59★	0.06
CI	0.02	0.02	0.15	0.02	-0.01	-0.01	0.05	0.00	0.26❖	-0.02
CTL	0.25❖	0.27❖	0.20❖	0.26❖	0.29❖	0.26❖	0.26❖	0.24❖	0.15	0.17
DA	0.47★	0.35❖	0.52★	0.33❖	0.47★	0.26❖	0.50★	0.25❖	0.49★	0.14
ECT	0.24❖	0.21❖	0.34❖	0.19	0.29❖	0.19	0.32❖	0.19	0.38❖	0.09
ET	0.51★	0.32❖	0.67◆	0.29❖	0.51★	0.23❖	0.57★	0.22❖	0.70◆	0.11
GF	0.15	0.15	0.18	0.18	0.16	0.23❖	0.18	0.23❖	0.16	0.19
LT	0.56★	0.34❖	0.65◆	0.28❖	0.52★	0.23❖	0.57★	0.21❖	0.62◆	0.07
MNT	0.40❖	0.26❖	0.44★	0.21❖	0.40❖	0.17	0.41★	0.14	0.43★	0.06
RA	0.05	0.05	0.16	0.02	0.11	0.00	0.13	0.00	0.22❖	-0.04
SE	0.28❖	0.14	0.38❖	0.09	0.28❖	0.02	0.31❖	0.03	0.40❖	-0.06
UT	0.14	0.15	0.16	0.11	0.28❖	0.09	0.25❖	0.09	0.14	0.02

labels: (▲) very strong, (◆) strong, (★) moderate, (❖) weak, () no correlation.

are counted, which reduces the correlation in relation to method coverage. In general, there is a moderate correlation between the smell AR and the test coverage.

The ET smell is related to a test method that verifies several functionalities. This smell has a strong correlation with complexity (0.67) and method coverage (0.7), as Figures 1(a) and (b) show, respectively. That is, for complexity coverage, the correlation indicates that to test the various paths in a class, several assertions are created in the same test method. Similarly, method coverage shows that the method has many assertions that verify various functionalities, which makes it difficult to understand the test code. According to Figures 1 (a) and (b), some of the test classes have about 50 ET and cover approximately 150 paths and 100 methods, respectively. Additionally, this smell test has a moderate correlation with the BranchC, InstructionC, and LineC.

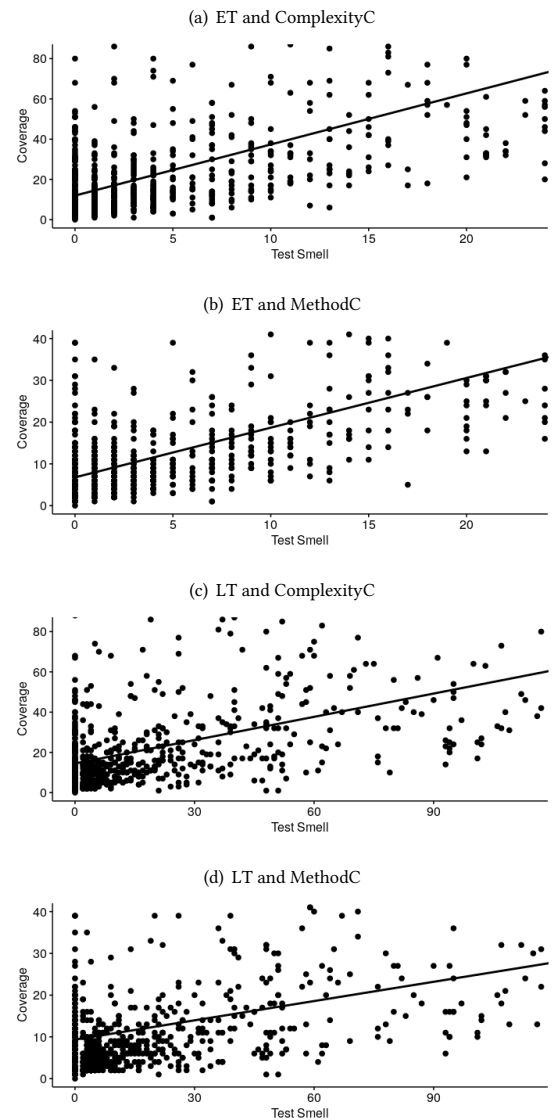
In contrast, LT is related to several test methods that only verify one method. LT correlates with all coverage metrics, reaching a weak to strong correlation, with the exception of MethodM metric. LT has a strong correlation with the coverages ComplexityC (0.65) and MethodC (0.62), as Figures 1 (c) e (d) show. Also, LT presents moderate correlation with BranchC, InstructionC and LineC.

The DA test smell is related to a method that checks the same condition several times within the same test method. DA has a weak to medium correlation with all coverage metrics, with the exception of the MethodM metric. DA metric also had its highest (moderate) correlation with ComplexityC (0.52). In addition, AR, ET, LT, and DA smells are related to duplicity and accomplishment of many tasks. For example, in the time that the DA smell occurs when duplicate assertions, an ET method checks several other methods, LT when several methods check one, and DA when the same condition is checked several times.

Finally, the missing code coverage metrics have either weak or no correlation with test smells. For example, among the missing test coverage metrics (BranchM, ComplexityM, InstructionM, LineM, and MethodM), all resulted in a weak correlation between 5 to 9 test smells, except the metric covering MethodM (it did not present correlation with any test smell).

6 THREATS TO VALIDITY

Internal Validity. In this study, a set of coverage metrics and another set of tests smells were selected from previous studies. These sets may contain other coverage metrics and test smells, besides the selected ones.

**Figure 1: Strong Correlations**

External Validity. In this study, we use free-code projects, which are real-world software, so this requirement is not a threat to external validity, since we can use other projects that pass the requirements cited above as input to this study. On the other hand, to reduce the scope of the study, only the JUnit library was used, because it is the library used in the referenced articles. Due to scope constraints, other programming languages that run on the Java Virtual Machine were not addressed in this study.

Conclusion Validity. In this experiment, the verification of the hypotheses was performed by checking the correlation, using Spearman’s correlation coefficient, between the test smells and the coverage metrics, from the data generated by the JNose Test tool.

7 RELATED WORK

Bavota et al. [1] investigated the diffusion of smells in test classes related to 27 software projects and their effects on software maintenance. In their work, ten types of test smells were analyzed. As a result, they found that test classes are affected by at least one smell. In addition, the presence of test smells has a negative impact on software maintenance.

Spadini et al. [17] investigated the relationship between test smells and their propensity for changes/defects through a set of one million test cases. In their work, six types of test smells were collected. As a result, they identified that the presence of test smells in test codes makes them more prone to changes and defects. Also, they identified that the production code is more prone to defects when tested by tests containing smells.

Palomba et al. [12] investigated how test smells refactoring can induce flaky tests correction, preventing them from hiding real bugs. In their work, three types of test smells were collected in 18 software systems. They found that most of the flaky tests contain test smells. In addition, they identified that the refactoring of test smells resulted in the removal of design flaws and the flaky tests correction caused by test smells.

Several approaches and tools have been proposed in the literature to identify test smells [2, 11, 12]. However, few studies focus on associating smells to software quality. Our work differs by investigating the correlation between “test smells” and “test coverage”. In this way, we value the influence of the “test smells” in the coverage of the projects, and we verify if this can influence the quality and the sustainability of the projects.

8 CONCLUDING REMARKS

In this paper, we analyzed the influence of test smells on code coverage. Thus, we selected eleven software projects, in which we collected twenty-one test smells and ten coverage metrics using the JNose Test, a tool developed to support this analysis. From our results, we observed that the Lazy Test stand out as the type of test smell that most occurred among all the anti-pattern studied, for all projects. Lazy Test is commonly caused by a large number of test methods related to only one method in the decay code. This situation indicates difficulties in maintaining the test [1, 2].

We calculated 210 correlations between test smells and test coverage metrics. As a result, we obtained 4 strong correlations, 17 moderate correlations, 42 weak correlations, 147 pairs without correlation. Based on data, we observed a relationship between

test smells and test coverage. In other words, test smells may influence test coverage. As future work, we intend to perform a more in-depth study of the reasons for each correlation of the 63 found. Furthermore, we intend to apply our study to more projects of different domains and sizes. We also plan to refine the JNose Test algorithm to identify the line of code in which test smells are found. In addition, we can analyze which parts of the code are most affected by test smells.

ACKNOWLEDGMENTS

This research was partially funded by INES 2.0; CNPq grants 465614/2014-0 and 408356/2018-9 and FAPESB grant JCB0060/2016.

REFERENCES

- [1] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? An empirical study. *Empirical Software Engineering* 20, 4 (2015).
- [2] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and David W. Binkley. 2012. An empirical analysis of the distribution of unit test smells and their impact on software maintenance. In *28th IEEE International Conference on Software Maintenance (ICSM)*.
- [3] David Bowes, Tracy Hall, Jean Petrić, Thomas Shippey, and Burak Turhan. 2017. How Good Are My Tests?. In *2017 IEEE/ACM 8th Workshop on Emerging Trends in Software Metrics (WETSoM)*.
- [4] Adalberto Nobiato Crespo, Odair J Silva, Carlos Alberto Borges, Clênio Figueiredo Salviano, M Argollo, and Mario Jino. 2004. Uma metodologia para teste de Software no Contexto da Melhoria de Processo. In *Simpósio Brasileiro de Qualidade de Software (SBQS)*.
- [5] Arie Deursen, Leon M.F. Moonen, A. Bergh, and Gerard Kok. 2001. *Refactoring Test Code*. Technical Report. Amsterdam, The Netherlands, The Netherlands.
- [6] Gerald D. Everett and Raymond McLeod Jr. 2007. *Automated Testing Tools* (1 ed.). Wiley-IEEE Computer Society Press.
- [7] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018).
- [8] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code Coverage for Suite Evaluation by Developers. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*.
- [9] Suhaimi Ibrahim and Shahid Muhammad. 2011. An Evaluation of Test Coverage Tools in Software Testing. In *International Conference On Telecommunication Technology And Application (ICTTA)*.
- [10] Gerard Meszaros. 2007. *xUnit test patterns: refactoring test code* (1st ed.). Addison-Wesley.
- [11] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the Diffusion of Test Smells in Automatically Generated Test Code: An Empirical Study. In *IEEE/ACM 9th International Workshop on Search-Based Software Testing (SBST)*.
- [12] Fabio Palomba, Andy Zaidman, and Andrea De Lucia. 2018. Automatic Test Smell Detection Using Information Retrieval Techniques. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*.
- [13] Anthony Peruma. 2018. What the Smell? An Empirical Investigation on the Distribution and Severity of Test Smells in Open Source Android Applications.
- [14] Roger Pressman. 2016. *Software Engineering: A Practitioner’s Approach* (8th ed.). McGraw-Hill, Inc., New York, NY.
- [15] Bart Van Rompaey, Bart Du Bois, Serge Demeyer, and Matthias Rieger. 2007. On The Detection of Test Smells: A Metrics-Based Approach for General Fixture and Eager Test. *IEEE Transactions on Software Engineering* 33, 12 (Dec 2007).
- [16] Neil J Salkind and Terese Rainwater. 2005. *Exploring research* (6 ed.). Prentice Hall.
- [17] Davide Spadini, Fabio Palomba, Andy Zaidman, Magiel Bruntink, and Alberto Bacchelli. 2018. On the Relation of Test Smells to Software Code Quality. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*.
- [18] Bharti Suri and Shweta Singhal. 2015. Investigating the OO characteristics of software using CKJM metrics. In *4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO) (Trends and Future Directions)*.
- [19] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An Empirical Investigation into the Nature of Test Smells. In *31st IEEE/ACM Intern. Conf. on Automated Software Engineering (ASE)*. ACM, New York, NY, USA.
- [20] Vahid Garousi Yusifoğlu, Yasaman Amannejad, and Aysu Betin Can. 2015. Software test-code engineering: A systematic mapping. *Information and Software Technology* 58 (2015).